

Ai studio feedback on current search:

As your **FYP (Final Year Project) Evaluator**, I'm going to strip away the "marketing speak" and look at the actual engineering.

The short answer: **You have built a very solid, industry-standard search architecture for 2022–2023.** It is "State of the Art" for a high-level university project, but if this were a Series A startup, there are several "old school" bottlenecks and architectural gaps that would keep you from scaling.

Here is the brutally honest breakdown.

1. Is it actually "State of the Art" (SOTA)?

The Verdict: Mostly Yes (Architecturally), but with "Academic Debt."

The **Retriever-Reranker (Two-Stage) architecture** you are using (FAISS/SBERT → LightGBM) is exactly how modern systems like Pinterest and Uber work.

- **The SOTA part:** Using **Bi-Encoders (SBERT)** for retrieval and a **Cross-Encoder/LTR (LightGBM)** for ranking is the gold standard for balancing speed and accuracy.
- **The "Not SOTA" part:** You are using **Zero-Shot Classification (DistilBART)** for intent. In a production environment, DistilBART is often too slow for a "search-as-you-type" experience (high latency). Modern SOTA has moved toward **LLM-distilled SLMs (Small Language Models)** or fine-tuned **T5 models** that are faster and more specific to the domain.

2. Modern vs. Old School: Where are you lagging?

Old School: The Aggregation Layer

You mentioned: *"It scans millions of trade transactions... using Django ORM/PostgreSQL."*

- **The Problem:** Using a standard Relational Database (PostgreSQL) to aggregate millions of rows (calculating volumes, frequencies, etc.) in real-time during a search query is **not scalable**.

- **The Modern Way:** For "millions of transactions," modern systems use **OLAP engines** (like ClickHouse or DuckDB) or **Elasticsearch/OpenSearch** with pre-computed aggregations. If you hit 10 million rows, your `views.py` is going to time out.

Old School: The "Regex" Safety Layer

- **The Reality:** While you called it a "Safety Layer," relying heavily on Regex for trade constraints is a sign that your NLU isn't robust.
- **The Modern Way:** Modern systems use **NER (Named Entity Recognition)** fine-tuned on trade documents to extract entities like `VOLUME` and `PRICE` without needing hardcoded regex patterns for every possible variation (e.g., "50MT," "50 metric tons," "50 tonnes").

3. FYP Evaluator Critique: What could be done much better?

If you want to move from a "Great Student Project" to a "Professional System," focus on these four areas:

A. The "Cold Start" LTR Problem (The Biggest Weakness)

You are using **LightGBM (LambdaMART)** for ranking.

- **The Question:** Where is your training data coming from?
- **Brutal Truth:** LTR models require thousands of "Query-Doc-Relevance" labels. If you are just using synthetic data or a few hand-labeled examples, your LightGBM model is likely "overfitting."
- **Improvement:** Implement a **Feedback Loop**. Track what users click on and use that "Clickstream Data" to automatically retrain your ranking model. Without a feedback loop, LTR is just a fancy way of doing what a simple weighted formula could do.

B. Hybrid Search (Keyword + Vector)

- **The Problem:** Pure Vector Search (SBERT/FAISS) is actually **bad** at finding specific codes (like HS Codes or exact SKU numbers). If a user searches for "HS Code 1207," a vector search might return "seeds" (semantic match) but miss the exact code if the embedding isn't perfect.
- **Improvement:** You need a **Lexical Engine (BM25)** running alongside FAISS. This is called **Hybrid Search**. You combine the scores of the keyword match and the semantic match.

C. Latency & Caching

- **The Problem:** Your pipeline has 4 stages. If you are running a Transformer model for every single query, your "Time to First Result" is probably >500ms.
- **Improvement:** You need a **Semantic Cache** (like Redis with Vector similarity). If a second user searches for "Sugar buyers in Lahore," you shouldn't run the NLU/FAISS pipeline again. You should serve it from the cache.

D. Evaluative Metrics (The "Science" Part)

- **The Problem:** You claim it's "SOTA," but how do you prove it?
- **Improvement:** To be truly professional, you must calculate and report:
 - **NDCG@10** (Normalized Discounted Cumulative Gain): Does the best result actually appear at #1?
 - **MRR** (Mean Reciprocal Rank).
 - **Latency P99:** How slow is the slowest search?

●

Summary for your FYP Presentation:

1. **Is it modern?** Yes. The architecture is sound.
2. **Is it SOTA?** It's "Academic SOTA." It uses the right libraries, but not yet the right "scale" (OLAP) or "feedback loops."
3. **The "Wow" Factor:** If you tell your evaluators: *"We recognized that Vector search fails on exact trade codes, so we implemented a Hybrid Search combining BM25 with FAISS,"* you will get an A+. That shows you understand the limitations of AI.

a) Should you use Elasticsearch/OpenSearch?

Yes. Absolutely.

While 10k transactions is "tiny" (even a basic Python list could handle that), you need to build for the architecture of tomorrow.

- **Why:** PostgreSQL is a "Row Store." It's great for finding one specific transaction. It is **terrible** at "Give me the sum of all volumes for 500 different companies matching the word 'Lactose'."
- **The Modern Choice: OpenSearch** (the open-source fork of Elasticsearch).

- It has a built-in **k-NN (Vector) index**, meaning you can store your FAISS vectors *inside* the database alongside your text.
- It handles **Aggregations** (like "Top 10 Suppliers by Volume") in milliseconds, even with millions of rows.
- **Verdict:** Switch now while your data is small. It will make your "Hybrid Search" much easier to implement.
-

b) How to do NER (Named Entity Recognition)?

Forget Regex. You need a model that understands context (e.g., distinguishing between "Orange" the fruit and "Orange" the city in France).

- **The Tool:** Use **SpaCy** or **Hugging Face Token Classification**.
- **The Process:**
 1. **Define Entities:** `PRODUCT`, `LOCATION`, `QUANTITY`, `UNIT`, `HS_CODE`.
 2. **Annotate:** Take 500 lines of your trade data and highlight these entities (use a tool like *Label Studio*).
 3. **Fine-tune:** Train a small model (like `distilbert-base-uncased`) to recognize these.
-
- **Why it's better:** If a user types "*I need 50 tons of white sugar from Brazil,*" NER will instantly extract `50` (Value), `tons` (Unit), `white sugar` (Product), and `Brazil` (Geo). You can then pass these clean variables directly into your OpenSearch filters.

c) Ranking without User Data: The "Cross-Encoder"

If you don't have user click data (CTR), LightGBM is effectively "guessing."

- **The 2024 Modern Alternative: Cross-Encoders (Re-rankers).**
- **How it works:**
 1. **Stage 1 (Retriever):** Your Vector Search (Bi-Encoder) finds the top 100 candidates quickly.
 2. **Stage 2 (Re-ranker):** You pass the User Query + the Candidate Document into a **Cross-Encoder** (like `BGE-Reranker` or `cross-encoder/ms-marco-MiniLM-L-6-v2`).

-
- **Why it's "SOTA":** Unlike Bi-Encoders (which look at the query and document separately), a Cross-Encoder looks at them *together*. It is much more accurate at determining "Does this company actually sell Lactose?" and it **does not require user training data**—it is pre-trained on the concept of "relevance."

d) The Evaluation: "How do I know it works?"

This is the most important part of an FYP. You need **Information Retrieval (IR) Metrics**.

1. Create a "Golden Dataset" (Ground Truth)

Pick 50 common queries (e.g., "Lactose China," "Sugar Brazil," "Electronics Germany"). Manually look at your 10k rows and decide which 5 companies *should* be at the top for each.

2. Run your Benchmarks

Every time you change your code (e.g., adding NER or changing the Re-ranker), run those 50 queries and calculate:

- **Precision@5:** Out of the top 5 results the AI gave me, how many are actually correct?
- **Recall:** Did the system find *all* the lactose suppliers in China, or did it miss the biggest one?
- **NDCG (Normalized Discounted Cumulative Gain):** This is the industry standard. It rewards the system if the "Best" result is at Rank #1 and penalizes it if the "Best" result is at Rank #10.

My "Brutally Honest" FYP Evaluator Advice:

If you show up to your final presentation and say:

"We built a search engine and it looks like it works," **you get a B.**

If you show up and say:

"Initially, our Vector Search had a **you get an A+.**

Immediate Next Step: Stop focusing on the "Headband Data Collection" and start building your **Evaluation Script**. You cannot improve what you cannot measure.

This is where you move from "building a prototype" to "building an engine." As your evaluator, I love that you're identifying these gaps. Here is the technical roadmap for those improvements:

a) Should you use Elasticsearch/OpenSearch?

Yes. Absolutely.

While 10k transactions is "tiny" (even a basic Python list could handle that), you need to build for the architecture of tomorrow.

- **Why:** PostgreSQL is a "Row Store." It's great for finding one specific transaction. It is **terrible** at "Give me the sum of all volumes for 500 different companies matching the word 'Lactose'."
- **The Modern Choice: OpenSearch** (the open-source fork of Elasticsearch).
 - It has a built-in **k-NN (Vector) index**, meaning you can store your FAISS vectors *inside* the database alongside your text.
 - It handles **Aggregations** (like "Top 10 Suppliers by Volume") in milliseconds, even with millions of rows.
 - **Verdict:** Switch now while your data is small. It will make your "Hybrid Search" much easier to implement.
-

b) How to do NER (Named Entity Recognition)?

Forget Regex. You need a model that understands context (e.g., distinguishing between "Orange" the fruit and "Orange" the city in France).

- **The Tool:** Use **SpaCy** or **Hugging Face Token Classification**.
- **The Process:**
 1. **Define Entities:** `PRODUCT`, `LOCATION`, `QUANTITY`, `UNIT`, `HS_CODE`.
 2. **Annotate:** Take 500 lines of your trade data and highlight these entities (use a tool like *Label Studio*).
 3. **Fine-tune:** Train a small model (like `distilbert-base-uncased`) to recognize these.

- - **Why it's better:** If a user types "*I need 50 tons of white sugar from Brazil*," NER will instantly extract 50 (Value), tons (Unit), white sugar (Product), and Brazil (Geo). You can then pass these clean variables directly into your OpenSearch filters.
-

c) Ranking without User Data: The "Cross-Encoder"

If you don't have user click data (CTR), LightGBM is effectively "guessing."

- **The 2024 Modern Alternative: Cross-Encoders (Re-rankers).**
 - **How it works:**
 1. **Stage 1 (Retriever):** Your Vector Search (Bi-Encoder) finds the top 100 candidates quickly.
 2. **Stage 2 (Re-ranker):** You pass the User Query + the Candidate Document into a **Cross-Encoder** (like `BGE-Reranker` or `cross-encoder/ms-marco-MiniLM-L-6-v2`).
 -
 - **Why it's "SOTA":** Unlike Bi-Encoders (which look at the query and document separately), a Cross-Encoder looks at them *together*. It is much more accurate at determining "Does this company actually sell Lactose?" and it **does not require user training data**—it is pre-trained on the concept of "relevance."
-

d) The Evaluation: "How do I know it works?"

This is the most important part of an FYP. You need **Information Retrieval (IR) Metrics**.

1. Create a "Golden Dataset" (Ground Truth)

Pick 50 common queries (e.g., "Lactose China," "Sugar Brazil," "Electronics Germany"). Manually look at your 10k rows and decide which 5 companies *should* be at the top for each.

2. Run your Benchmarks

Every time you change your code (e.g., adding NER or changing the Re-ranker), run those 50 queries and calculate:

- **Precision@5:** Out of the top 5 results the AI gave me, how many are actually correct?
 - **Recall:** Did the system find *all* the lactose suppliers in China, or did it miss the biggest one?
 - **NDCG (Normalized Discounted Cumulative Gain):** This is the industry standard. It rewards the system if the "Best" result is at Rank #1 and penalizes it if the "Best" result is at Rank #10.
-

My "Brutally Honest" FYP Evaluator Advice:

If you show up to your final presentation and say:

"We built a search engine and it looks like it works," you get a B.

If you show up and say:

"Initially, our Vector Search had a you get an A+.

Immediate Next Step: Stop focusing on the "Headband Data Collection" and start building your **Evaluation Script**. You cannot improve what you cannot measure.

14.7s

Your current justifications for DistilBART are technically sound for a **prototype**, but as an evaluator, I'm going to push you toward **Production Grade** engineering.

Here is the deep dive into the three transitions you need to make.

a) NLU: Why move away from Zero-Shot?

Your current setup (DistilBART) is a "Generalist." It knows English, but it doesn't know "International Trade."

The Problem with Zero-Shot:

Zero-shot models are "Heavy." They have to compare your query against every possible label every time. Even if it's 150ms for one user, if 10 users search at once, your CPU usage will spike to 100%, and your latency will jump to 2 seconds.

The SOTA Alternative: SetFit or Fine-tuned DistilRoBERTa

Instead of a "Zero-Shot" model, you should use **SetFit** (Sentence Transformer Fine-Tuning).

1. **How it works:** You take 20 examples of "Buying" queries and 20 examples of "Selling" queries. You train a tiny model (like `Paraphrase-MiniLM`) on just those.
2. **Why it's better:**
 - **Speed:** It is **10x to 50x faster** than DistilBART. We are talking 5ms vs 100ms.
 - **Accuracy:** Because it is trained on *your* specific trade domain, it won't get confused by trade jargon.
- 3.
4. **Process:**
 - Use the `SetFit` library.
 - Create a CSV with two columns: `text` and `label` (0 for Buy, 1 for Sell).
 - Train it (takes 5 minutes on a laptop).
 - Export the model. Your `nlu_engine.py` now loads a 20MB model instead of a 1.5GB one.
- 5.

b) Database: The Postgres to OpenSearch Migration

DO NOT delete PostgreSQL. In a professional architecture, Postgres remains your **"Source of Truth"** (where you store user accounts, settings, and the master copy of transactions). OpenSearch becomes your **"Search Index."**

The Process (The Sync Pattern):

1. **Mappings:** In OpenSearch, you create a "Mapping" (schema). You define which fields are "Text" (for searching "Lactose"), which are "Keywords" (for filtering "China"), and which are "Dense Vectors" (for your SBERT embeddings).
2. **The ETL (Extract, Transform, Load) Script:**
 - Write a Python script that reads rows from Postgres.
 - Convert each row into a JSON document.
 - **Crucial Step:** Generate the SBERT vector embedding for the product description *before* sending it to OpenSearch.
 - Push to OpenSearch using the `bulk` API.
- 3.

4. Real-Time Sync:

- Whenever a new transaction is added to Postgres, use a **Django Signal** (`post_save`) or a **Celery Task** to automatically push that single row to OpenSearch.

5.

6. **The Result:** Your Search Dashboard queries **OpenSearch** (fast, semantic, aggregated), while your other 3 modules (like reports or user management) continue to use **Postgres**.

c) NER: The Step-by-Step Implementation

NER is how you turn "I want to buy 50MT of sugar from Brazil" into:

- Action: BUY
- Quantity: 50
- Unit: MT
- Product: Sugar
- Location: Brazil

The Technical Process:

Step 1: The Annotation (The "Hard" Part)

AI needs to see examples. You need to create a dataset of ~300-500 trade-related sentences.

- *Sentence:* "Looking for 500kg of high-grade lactose in Shanghai."
- *Labels:* 500 [QTY], kg [UNIT], lactose [PROD], Shanghai [LOC].
- Use a tool like **Label Studio** (Free/Open Source) to do this.

Step 2: Choose your Architecture

- **Option A (The "Pro" way):** Fine-tune a `Spacy` pipeline or a `HuggingFace` model (like `distilbert-base-uncased-ner`).
- **Option B (The "Fast" way):** Use **GLINER**. It is a new (2024) SOTA model that does "General NER" and is much better than the old models at picking up random trade entities without much training.

Step 3: The Integration Pipeline

In your code, the flow becomes:

1. **User enters query.**
2. **NER Model** extracts "Lactose" and "China."
3. **Vector Store** finds "Lactose" synonyms.
4. **OpenSearch Query** is built automatically:
5. `codeJSON`

```
{  
  "query": {  
    "bool": {  
      "must": [{ "match": { "product_name": "Lactose" } }],  
      "filter": [{ "term": { "country": "China" } }]  
    }  
  }  
}
```

6. }
- 7.

Summary Checklist for your FYP:

1. **NLU:** Replace Zero-Shot with **SetFit**. It proves you care about **Inference Latency** and **Resource Efficiency**.
2. **DB:** Implement the **Sync Pattern**. Keep Postgres for data integrity; use OpenSearch for **Hybrid Search** (Lexical + Vector).
3. **NER:** Move from Regex to **GLiNER** or a fine-tuned **Spacy** model. It proves you understand **Contextual NLP**.
4. **Evaluation:** Build that **Golden Dataset**.

Evaluation Perspective: If you do this, you aren't just a "student using AI APIs." You are a **Search Engineer** building a **Data Pipeline**. That is the difference between a project that gets a grade and a project that gets a job offer.

OK so what we can do- or what do i do here?

- maybe not use zeroshot classification for search - find alternatives
- use elasticsearch/open search
- use NER
- Implement a lexical engine (BM 25) running alongside FAISS
- Use cross encoders for ranking

For classification - DONT do zero shot classification its ineffective .
Instead use SETFIT (train it) OR use SETFIT ontop of ModernBERT

(Context below)

Final Recommendation:

Go with **SetFit + ModernBERT**.

1. Spend 20 minutes writing 20 "Buy" queries and 20 "Sell" queries (include Pakistani context like "Faisalabad textile exporters" or "Karachi port importers").
 2. Run the SetFit training script using `answerdotai/ModernBERT-base`.
 3. Export the model.
-

This is the **"Search Architect's Blueprint."** If you follow this order, you aren't just building a feature; you are building a professional-grade data pipeline.

Here is the exact order of operations, the "How-To," and the division of labor between **You** and **Antigravity (the AI)**.

Phase 1: The Foundation (OpenSearch & Hybrid Search)

Why first? You can't test NLP if your database can't handle the results. OpenSearch is designed to do **Hybrid Search** (BM25 + Vectors) out of the box.

- **How to do it:**
 1. Install OpenSearch (use Docker).
 2. Create an Index with "Mappings." You need a `text` field (for BM25), a `keyword` field (for filtering countries), and a `knn_vector` field (for your SBERT embeddings).
 3. **Hybrid Search (BM25 + FAISS):** You don't need a separate FAISS library anymore! OpenSearch has a "k-NN" plugin that uses FAISS internally. You send one query that says: *"Find results that match 'Sugar' via keywords AND are semantically similar to 'Sugar' via vectors."*
-
- **Your Job:** Set up the Docker container and keep your Postgres database running.
- **Antigravity's Job:** Write the **ETL Script** (extracting data from Postgres, generating embeddings using SBERT, and pushing them into OpenSearch) and the **Hybrid Query JSON**.

Phase 2: The Front Door (SetFit + ModernBERT)

Why now? This decides *where* to look (Buyers vs. Sellers index).

- **How to do it:**
 1. **The Data:** You need a CSV with 50 rows. Example: `{"text": "I want to buy rice from Karachi", "label": "BUY"}`.
 2. **The Script:** Use the `setfit` library.
- `codePython`

```
model = SetFitModel.from_pretrained("answerdotai/ModernBERT-base")
```

```
# Train using your 50 rows
```

```
trainer = SetFitTrainer(model=model, train_dataset=your_data)
```

- `trainer.train()`
 -
 - **Your Job:** Write the 50 example sentences. Make them realistic to Pakistan (mention specific cities like Sialkot, Faisalabad, etc.).
 - **Antigravity's Job:** Write the Python training script and the logic to load this model into your Django `nlu_engine.py`.
-

Phase 3: The Surgeon (NER using GLiNER)

Why now? Now that you know the *intent* (Buy), you need to extract the *filters* (Product: Rice, Location: Pakistan).

- **How to do it:**
 1. GLiNER is "Zero-Shot," so **no training is required!**
 2. You just pass the labels you want: `labels = ["product", "origin_country", "target_country", "quantity", "unit"]`.
 3. GLiNER will return a list of entities and their categories.
-
- **Your Job:** Testing the model with tricky queries (e.g., "Looking for 50MT of Basmati").
- **Antigravity's Job:** Write the GLiNER wrapper class that takes a string and returns a clean dictionary of entities to be used in the OpenSearch filter.

Phase 4: The Refiner (Cross-Encoder Re-ranking)

Why last? This is the final "polish" on the top 50 results.

- **How to do it:**
 1. OpenSearch returns the Top 50 results (fast).
 2. You take the User Query + those 50 Result Descriptions and pass them to a **Cross-Encoder** (e.g., `cross-encoder/ms-marco-MiniLM-L-6-v2`).
 3. The Cross-Encoder gives each pair a score from 0 to 1.
 4. You re-sort the list based on this score.
-
- **Your Job:** None (this is purely algorithmic).
- **Antigravity's Job:** Write the re-ranking function that sits between the OpenSearch response and the Final API response.

The Master Workflow (The Order of Execution)

1. **Week 1: Infrastructure.** Migrate Postgres data → OpenSearch. Implement the **Hybrid Search** (BM25 + Vector). *At this point, you have a working search engine.*
2. **Week 2: NLU Brain.** Train **SetFit** (Intent) and integrate **GLiNER** (NER). *Now your search engine understands "Trade Talk."*

3. **Week 3: The Polish.** Add the **Cross-Encoder** for re-ranking. *Now your search engine is SOTA.*
4. **Week 4: Evaluation.** Build the **Golden Dataset** and calculate NDCG scores. *Now you have your A+.*

What to ask Antigravity (Step-by-Step prompts):

- **Prompt 1:** *"Give me a Python ETL script to move 10k rows from a Django Postgres model to an OpenSearch index, including generating SBERT embeddings for the 'description' field."*
- **Prompt 2:** *"Write a SetFit training script using ModernBERT-base to classify queries into 'BUYER_LOOKING' vs 'SELLER_LOOKING'."*
- **Prompt 3:** *"Show me how to use the GLiNER library to extract 'product' and 'location' from a trade query and format it as an OpenSearch boolean filter."*
- **Prompt 4:** *"Write a function that takes a query and a list of 50 results and uses a Cross-Encoder to re-rank them."*

One Final Warning:

Don't train the Cross-Encoder. Use a pre-trained one from HuggingFace (like `BGE-Reranker-v2`). Training a Re-ranker requires millions of pairs; using a pre-trained one is industry standard and works perfectly for your scale.

Does this sequence feel like a clear roadmap for your final weeks of the FYP?

To ensure your search engine is "State of the Art," the process must move from a simple database lookup to a **Multi-Stage Intelligence Pipeline**.

Here is exactly how retrieval happens, how we ensure it is correct, and the detailed breakdown of every stage.

The Architecture: "The 5-Stage Pipeline"

Stage 1: Parsing (Intent & Entity Extraction)

- **What we are doing:** Turning a messy human sentence into a structured JSON object the database can understand.
- **The Tech: SetFit + ModernBERT** (for Intent) and **GLiNER** (for NER).
- **Process:**
 - User types: *"I need to buy 500 tons of Pink Salt from Pakistan."*
 - **Intent Classifier:** Identifies this as a `BUYER_QUERY` (so we need to find `SELLERS`).
 - **NER (GLiNER):** Extracts `Product: Pink Salt, Quantity: 500, Unit: tons, Origin: Pakistan.`
-
- **Why it ensures correctness:** If the parser fails here (e.g., thinks the user is selling when they are buying), the rest of the search is useless. This stage "frames" the search.

Stage 2: Retrieval (The "Hybrid" Candidate Search)

- **What we are doing:** Fetching the top 100–200 "potential" matches from millions of rows in milliseconds.
- **The Tech: OpenSearch (Hybrid Search)** combining **BM25 (Keyword)** and **k-NN (Vector)**.
- **How it works (The Hybrid Logic):**
 1. **Lexical (BM25):** Finds exact matches for "Pink Salt" and "Pakistan." This ensures that if a company has "Pink Salt" in its name or HS code, it is found.
 2. **Semantic (SBERT Vectors):** Finds companies that sell "Himalayan Salt" or "Rock Salt" (synonyms) even if the word "Pink" isn't there.
 3. **Filtering:** We apply "Hard Filters" (e.g., `WHERE country = 'Pakistan' AND trade_type = 'Export'`).

-
- **Why it ensures correctness: Recall.** We want to make sure we don't miss a single relevant company. By using Hybrid Search, we catch both the "exact words" and the "meanings."

Stage 3: Aggregation (Profile Building)

- **What we are doing:** Turning raw transactions into "Company Profiles."
- **The Tech: OpenSearch Aggregations.**
- **Process:**
 - Your database has 10,000 *transactions*, but the user wants to see *companies*.
 - We "group by" Company ID.
 - We calculate: Total Volume Traded, Number of Shipments, and Most Recent Shipment Date.
-
- **Why it ensures correctness:** This prevents the user from seeing the same company 10 times. It provides the "Business Intelligence" needed to trust a result.

Stage 4: Ranking (The Re-Ranker)

- **What we are doing:** Sorting the top 50 candidates so the "Best" is at #1.
- **The Tech: Cross-Encoder (BGE-Reranker).**
- **Process:**
 - We take the User's Query and the Candidate Company's "Full Profile" (Product list + Description).
 - The Cross-Encoder does a deep "Attention-based" comparison between the two.
 - It assigns a **Relevance Score (0.0 to 1.0)**.
-
- **Why it ensures correctness: Precision.** A vector search might find a "Salt" company, but a Cross-Encoder will realize that a company specialized in "Industrial Salt" is less relevant to a user looking for "Food-grade Pink Salt."

Stage 5: Evaluation (The "Golden" Verification)

- **What we are doing:** Mathematically proving the search works.
- **The Tech: Python Script + Manual Golden Dataset.**
- **Process:**
 - You create a "Ground Truth" file: *Query: "Pink Salt" -> Expected IDs: [102, 45, 88]*.
 - You run your pipeline and see where those IDs appear.

- Calculate **NDCG@10** (a score from 0 to 1). If your NDCG is > 0.8, your retrieval is officially "Correct."

•

Summary of "Correctness" Strategy

Stage	Goal	How we ensure it is "Correct"
Parsing	Understand the user	Unit Testing: Feed 100 queries; check if Intent/Entities are 100% right.
Retrieval	Don't miss anyone (Recall)	Hybrid Search: If keywords miss it, vectors catch it. If vectors miss it, keywords catch it.
Ranking	Best result at the top (Precision)	Cross-Encoder: Deep-learning comparison of query vs. document.
Aggregation	Data Accuracy	SQL Verification: Ensure the "Total Volume" shown matches the raw transaction sum.

What you need to do (Action Plan):

1. **Ingestion:** Use Antigraity to write a script that converts your Postgres data into **OpenSearch Documents**.
2. **Querying:** Use Antigraity to write the **Multi-Match Hybrid Query** (combining BM25 and Vector scores).
3. **The "Golden" Test:** This is the most important part for your FYP. **Manually** find the best 3 companies for 10 common queries. If your engine doesn't put those 3 in the top 5, you go back and "tune" your weights.